

# DATA ANALYTICS ASSIGNMENT – CASE STUDY

By  
K.RUTHIKA REDDY-245805344

Steps to do:

## 1. Data collection :

First we collected a dataset from Kaggle for our case study. We took the cereals dataset.

Dataset : [C:\Users\sripr\Downloads\cereal 1 \(1\).csv](C:\Users\sripr\Downloads\cereal 1 (1).csv)

## 2. Data Preprocessing :

Handle missing values, fix data types, remove duplicates, normalize/scaling,  
Encode if needed.

## 3. Data Visualization & Exploration :

- Explore patterns, trends, distributions, and relationships.
- In this step first we found size and structure of the dataset and found how categorical variables are distributed.
- Here we used correlation heat map, histograms, box plots, pair plots, and bar charts for categorical variables to visualise the data.

## 4. Statistical Analysis / Hypothesis Testing :

- **Statistical analysis** helps summarize and interpret data to uncover meaningful patterns, trends, and relationships.
- **Hypothesis testing** is a key part of this process—it evaluates whether observed differences or correlations in the dataset are statistically significant or occurred by random chance.
- Here we are performing 2 hypotheses tests.

## 5. Interpretation & Reporting

This stage involves analyzing the statistical results and visual findings to draw meaningful conclusions from the data. It helps summarize key insights, validate hypotheses, and present the overall outcomes of the analysis in a clear and concise manner.

# DATA PREPROCESSING ( cleaning)

INPUT CODE:

```
[3]: #DATA PREPROCESSING
import pandas as pd
import numpy as np

# 1 Load dataset
df = pd.read_csv(r"C:\Users\sripri\Downloads\cereal 1 (1).csv")

# 2 Basic info
print("\n Dataset Info:")
print(df.info())
print("\nFirst 5 Rows:")
print(df.head())

# 3 Handle missing values
print("\nMissing values before cleaning:")
print(df.isnull().sum())

# Fill missing numeric values with mean, categorical with mode
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].mean())

print("\nMissing values after filling:")
print(df.isnull().sum())

# 4 Remove duplicates
df.drop_duplicates(inplace=True)

# 5 Convert data types (where possible)
for col in df.columns:
    try:
        df[col] = pd.to_numeric(df[col])
    except:
        pass # ignore if column can't be converted (like cereal names)

# 6 Encode categorical columns manually
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    df[col] = df[col].astype('category').cat.codes
    # assigns unique int codes

# 7 Normalize numerical columns (min-max scaling manually)
num_cols = df.select_dtypes(include=[np.number]).columns
df[num_cols] = (df[num_cols] - df[num_cols].min()) / (df[num_cols].max() - df[num_cols].min())

# 8 Show cleaned data
print("\n Cleaned Data Sample:")
print(df.head(10))

# 9 Save cleaned data
df.to_csv(r"C:\Users\sripri\Downloads\cleaned_cereal 1 (1).csv", index=False)
print("\nCleaned dataset saved as 'cereal_cleaned.csv'")
```

Here at the last cleaned cereal data set is stored as cleaned\_cereal

Link for cleaned data set: [C:\Users\sripri\Downloads\cleaned\\_cereal 1 \(1\).csv](C:\Users\sripri\Downloads\cleaned_cereal 1 (1).csv)

OUTPUT:

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74 entries, 0 to 73
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Cereal Name                           74 non-null     object
1   Manufacturer                           74 non-null     object
2   Type                                  74 non-null     object
3   Calories                              74 non-null     int64
4   Protein (g)                           74 non-null     int64
5   Fat                                    74 non-null     int64
6   Sodium                                74 non-null     int64
7   Dietary Fiber                          74 non-null     float64
8   Carbs                                 74 non-null     float64
9   Sugars                                74 non-null     int64
10  Display Shelf                          74 non-null     int64
11  Potassium                              74 non-null     int64
12  Vitamins and Minerals                  74 non-null     int64
13  Serving Size Weight                    74 non-null     float64
14  Cups per Serving                       74 non-null     float64
dtypes: float64(4), int64(8), object(3)
memory usage: 8.8+ KB
None

First 5 Rows:
      Cereal Name      Manufacturer Type  Calories  Protein (g)  Fat  \
0      100%_Bran      Nabisco      C      70      4      1
1  100%_Natural_Bran  Quaker Oats  C      120      3      5
2      All-Bran      Kelloggs      C      70      4      1
3  All-Bran_with_Extra_Fiber  Kelloggs  C      50      4      0
4      Almond_Delight  Ralston Purina  C      110      2      2

      Sodium  Dietary Fiber  Carbs  Sugars  Display Shelf  Potassium  \
0      130      10.0      5.0      6      3      280
1      15      2.0      8.0      8      3      135
2      260      9.0      7.0      5      3      320
3      140      14.0      8.0      0      3      330
4      200      1.0      14.0      8      3      -1

      Vitamins and Minerals  Serving Size Weight  Cups per Serving
0      25      1.0      0.33
1      0      1.0      -1.00
2      25      1.0      0.33
3      25      1.0      0.50
4      25      1.0      0.75

Missing values before cleaning:
Cereal Name      0
Manufacturer      0
Type              0
Calories          0
Protein (g)       0
Fat              0
Sodium            0
Dietary Fiber     0
Carbs             0
Sugars            0
Display Shelf     0
Potassium         0
Vitamins and Minerals  0
Serving Size Weight  0
Cups per Serving  0
dtype: int64
```

Missing values after filling:

```
Cereal Name      0
Manufacturer     0
Type            0
Calories        0
Protein (g)     0
Fat             0
Sodium         0
Dietary Fiber   0
Carbs          0
Sugars         0
Display Shelf   0
Potassium       0
Vitamins and Minerals 0
Serving Size Weight 0
Cups per Serving 0
dtype: int64
```

Cleaned Data Sample:

|   | Cereal Name | Manufacturer | Type | Calories | Protein (g) | Fat | Sodium \ |
|---|-------------|--------------|------|----------|-------------|-----|----------|
| 0 | 0.000000    | 0.500000     | 0.0  | 0.181818 | 0.6         | 0.2 | 0.406250 |
| 1 | 0.013699    | 0.833333     | 0.0  | 0.636364 | 0.4         | 1.0 | 0.046875 |
| 2 | 0.027397    | 0.333333     | 0.0  | 0.181818 | 0.6         | 0.2 | 0.812500 |
| 3 | 0.041096    | 0.333333     | 0.0  | 0.000000 | 0.6         | 0.0 | 0.437500 |
| 4 | 0.054795    | 1.000000     | 0.0  | 0.545455 | 0.2         | 0.4 | 0.625000 |
| 5 | 0.068493    | 0.166667     | 0.0  | 0.545455 | 0.2         | 0.4 | 0.562500 |
| 6 | 0.082192    | 0.333333     | 0.0  | 0.545455 | 0.2         | 0.0 | 0.390625 |
| 7 | 0.095890    | 0.166667     | 0.0  | 0.727273 | 0.4         | 0.4 | 0.656250 |
| 8 | 0.109589    | 1.000000     | 0.0  | 0.363636 | 0.2         | 0.2 | 0.625000 |
| 9 | 0.123288    | 0.666667     | 0.0  | 0.363636 | 0.4         | 0.0 | 0.656250 |

|   | Dietary Fiber | Carbs    | Sugars | Display Shelf | Potassium \ |
|---|---------------|----------|--------|---------------|-------------|
| 0 | 0.714286      | 0.250000 | 0.4375 | 1.0           | 0.848943    |
| 1 | 0.142857      | 0.375000 | 0.5625 | 1.0           | 0.410876    |
| 2 | 0.642857      | 0.333333 | 0.3750 | 1.0           | 0.969789    |
| 3 | 1.000000      | 0.375000 | 0.0625 | 1.0           | 1.000000    |
| 4 | 0.071429      | 0.625000 | 0.5625 | 1.0           | 0.000000    |
| 5 | 0.107143      | 0.479167 | 0.6875 | 0.0           | 0.214502    |
| 6 | 0.071429      | 0.500000 | 0.9375 | 0.5           | 0.093656    |
| 7 | 0.142857      | 0.791667 | 0.5625 | 1.0           | 0.305136    |
| 8 | 0.285714      | 0.666667 | 0.4375 | 0.0           | 0.380665    |
| 9 | 0.357143      | 0.583333 | 0.3750 | 1.0           | 0.577039    |

|   | Vitamins and Minerals | Serving Size Weight | Cups per Serving |
|---|-----------------------|---------------------|------------------|
| 0 | 0.25                  | 0.50                | 0.532            |
| 1 | 0.00                  | 0.50                | 0.000            |
| 2 | 0.25                  | 0.50                | 0.532            |
| 3 | 0.25                  | 0.50                | 0.600            |
| 4 | 0.25                  | 0.50                | 0.700            |
| 5 | 0.25                  | 0.50                | 0.700            |
| 6 | 0.25                  | 0.50                | 0.800            |
| 7 | 0.25                  | 0.83                | 0.700            |
| 8 | 0.25                  | 0.50                | 0.668            |
| 9 | 0.25                  | 0.50                | 0.668            |

Cleaned dataset saved as 'cereal\_cleaned.csv'

## Data Visualization & Exploration

```
10]: # Exploratory data analysis
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load the cleaned dataset
df = pd.read_csv(r"C:\Users\sripri\Downloads\cleaned_cereal 1 (1).csv")

# Basic structure and overview
print("\n Dataset Shape:", df.shape)
print("\n Column Names:", df.columns.tolist())
print("\n Data Types:\n", df.dtypes)
print("\n First 5 Rows:\n", df.head())

# Summary statistics
print("\n Summary Statistics:\n", df.describe().T)

# Check for missing values and duplicates
print("\n Missing Values:\n", df.isnull().sum())
print("\n Duplicate Rows:", df.duplicated().sum())

# Unique values in categorical columns
cat_cols = df.select_dtypes(include=['object', 'category']).columns
for col in cat_cols:
    print(f"\n Unique values in '{col}': {df[col].unique()}")

# Correlation matrix (for numeric columns)
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# Distribution of numeric columns
num_cols = df.select_dtypes(include=[np.number]).columns
df[num_cols].hist(figsize=(12,10), bins=15, color='skyblue', edgecolor='black')
plt.suptitle("Distribution of Numeric Features")
plt.show()

# Boxplots to detect outliers
df[num_cols].plot(
    kind='box',
    subplots=True,
    layout=(4,4),      # enough space for up to 16 variables
    figsize=(15,12),
    sharex=False,
    sharey=False
)
plt.suptitle("Boxplots for Outlier Detection", y=1.02)
plt.tight_layout()
plt.show()

# Pairplot for relationships between numeric variables
sns.pairplot(df[num_cols])
plt.suptitle("Pairplot of Numeric Variables", y=1.02)
plt.show()

# Feature-wise insights
for col in num_cols:
    print(f"\n {col} → mean: {df[col].mean():.3f}, median: {df[col].median():.3f}, std: {df[col].std():.3f}")

# Optional: Value counts for categorical columns
for col in cat_cols:
    plt.figure(figsize=(8,4))
    df[col].value_counts().plot(kind='bar', color='lightgreen', edgecolor='black')
    plt.title(f"Value Counts for {col}")
    plt.xlabel(col)
    plt.ylabel("Count")
    plt.show()
```

OUTPUT:

```
Dataset Shape: (74, 15)

Column Names: ['Cereal Name', 'Manufacturer', 'Type', 'Calories', 'Protein (g)', 'Fat', 'Sodium', 'Dietary Fiber', 'Carbs', 'Sugars', 'Display Shelf', 'Potassium', 'Vitamins and Minerals', 'Serving Size Weight', 'Cups per Serving']

Data Types:
Cereal Name      float64
Manufacturer     float64
Type             float64
Calories         float64
Protein (g)      float64
Fat              float64
Sodium           float64
Dietary Fiber    float64
Carbs            float64
Sugars           float64
Display Shelf    float64
Potassium        float64
Vitamins and Minerals float64
Serving Size Weight float64
Cups per Serving float64
dtype: object

First 5 Rows:
Cereal Name  Manufacturer  Type  Calories  Protein (g)  Fat  Sodium \
0    0.000000    0.500000  0.0  0.181818    0.6  0.2  0.406250
1    0.013699    0.833333  0.0  0.636364    0.4  1.0  0.046875
2    0.027397    0.333333  0.0  0.181818    0.6  0.2  0.812500
3    0.041096    0.333333  0.0  0.000000    0.6  0.0  0.437500
4    0.054795    1.000000  0.0  0.545455    0.2  0.4  0.625000

Dietary Fiber  Carbs  Sugars  Display Shelf  Potassium \
0    0.714286  0.250000  0.4375    1.0  0.848943
1    0.142857  0.375000  0.5625    1.0  0.410876
2    0.642857  0.333333  0.3750    1.0  0.969789
3    1.000000  0.375000  0.0625    1.0  1.000000
4    0.071429  0.625000  0.5625    1.0  0.000000

Vitamins and Minerals  Serving Size Weight  Cups per Serving
0          0.25          0.5          0.532
1          0.00          0.5          0.000
2          0.25          0.5          0.532
3          0.25          0.5          0.600
4          0.25          0.5          0.700

Summary Statistics:
count      mean      std  min      25%      50% \
Cereal Name  74.0  0.500000  0.294600  0.0  0.250000  0.500000
Manufacturer  74.0  0.436937  0.278207  0.0  0.166667  0.333333
Type          74.0  0.040541  0.198569  0.0  0.000000  0.000000
```



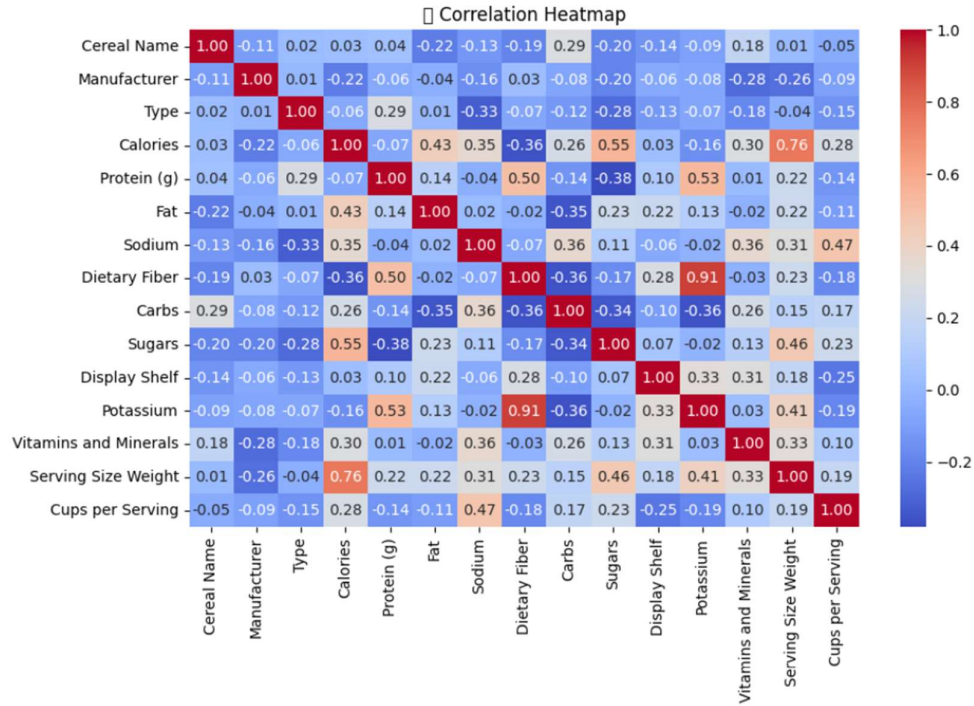
|                       |      |          |          |     |          |          |
|-----------------------|------|----------|----------|-----|----------|----------|
| Type                  | 74.0 | 0.040541 | 0.198569 | 0.0 | 0.000000 | 0.000000 |
| Calories              | 74.0 | 0.504914 | 0.167656 | 0.0 | 0.454545 | 0.545455 |
| Protein (g)           | 74.0 | 0.300000 | 0.217709 | 0.0 | 0.200000 | 0.200000 |
| Fat                   | 74.0 | 0.189189 | 0.192722 | 0.0 | 0.000000 | 0.200000 |
| Sodium                | 74.0 | 0.502111 | 0.266214 | 0.0 | 0.410156 | 0.562500 |
| Dietary Fiber         | 74.0 | 0.149324 | 0.171714 | 0.0 | 0.017857 | 0.107143 |
| Carbs                 | 74.0 | 0.649775 | 0.181217 | 0.0 | 0.541667 | 0.625000 |
| Sugars                | 74.0 | 0.485642 | 0.279230 | 0.0 | 0.250000 | 0.437500 |
| Display Shelf         | 74.0 | 0.587838 | 0.416756 | 0.0 | 0.000000 | 0.500000 |
| Potassium             | 74.0 | 0.283008 | 0.213227 | 0.0 | 0.123867 | 0.274924 |
| Vitamins and Minerals | 74.0 | 0.283784 | 0.227871 | 0.0 | 0.250000 | 0.250000 |
| Serving Size Weight   | 74.0 | 0.527432 | 0.151260 | 0.0 | 0.500000 | 0.500000 |
| Cups per Serving      | 74.0 | 0.651622 | 0.230737 | 0.0 | 0.668000 | 0.700000 |

|                       | 75%      | max |
|-----------------------|----------|-----|
| Cereal Name           | 0.750000 | 1.0 |
| Manufacturer          | 0.666667 | 1.0 |
| Type                  | 0.000000 | 1.0 |
| Calories              | 0.545455 | 1.0 |
| Protein (g)           | 0.400000 | 1.0 |
| Fat                   | 0.200000 | 1.0 |
| Sodium                | 0.679688 | 1.0 |
| Dietary Fiber         | 0.214286 | 1.0 |
| Carbs                 | 0.750000 | 1.0 |
| Sugars                | 0.734375 | 1.0 |
| Display Shelf         | 1.000000 | 1.0 |
| Potassium             | 0.346677 | 1.0 |
| Vitamins and Minerals | 0.250000 | 1.0 |
| Serving Size Weight   | 0.500000 | 1.0 |
| Cups per Serving      | 0.800000 | 1.0 |

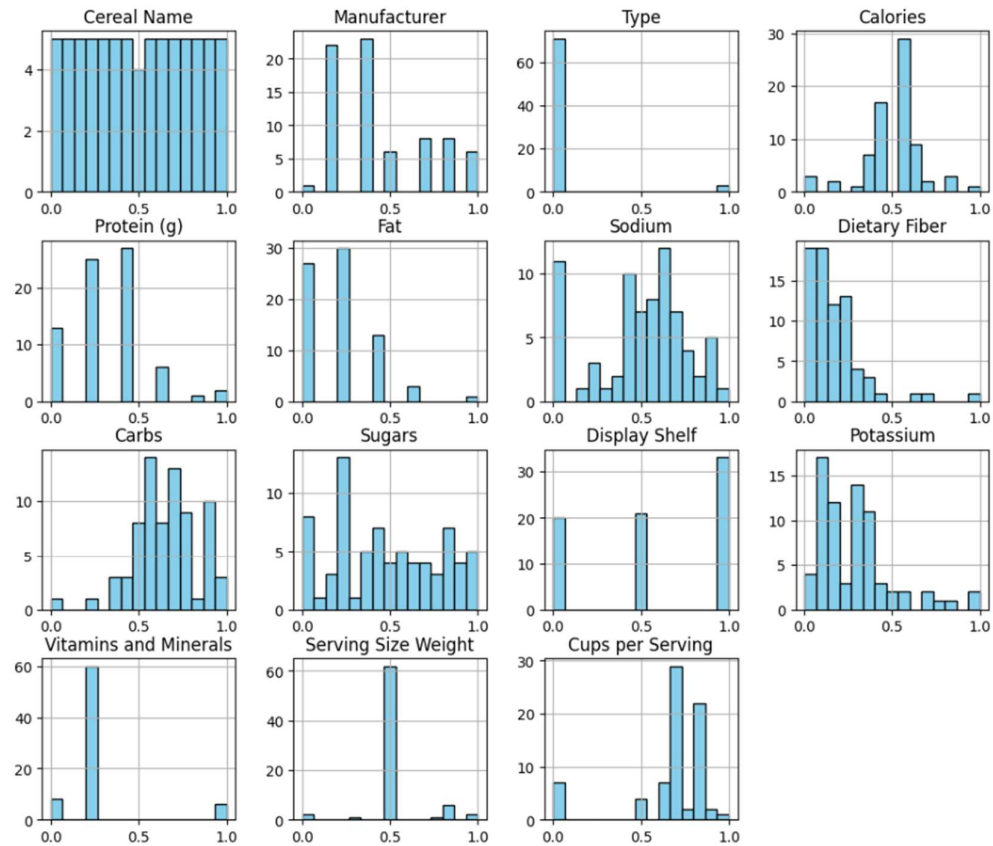
Missing Values:

|                       |   |
|-----------------------|---|
| Cereal Name           | 0 |
| Manufacturer          | 0 |
| Type                  | 0 |
| Calories              | 0 |
| Protein (g)           | 0 |
| Fat                   | 0 |
| Sodium                | 0 |
| Dietary Fiber         | 0 |
| Carbs                 | 0 |
| Sugars                | 0 |
| Display Shelf         | 0 |
| Potassium             | 0 |
| Vitamins and Minerals | 0 |
| Serving Size Weight   | 0 |
| Cups per Serving      | 0 |

dtype: int64



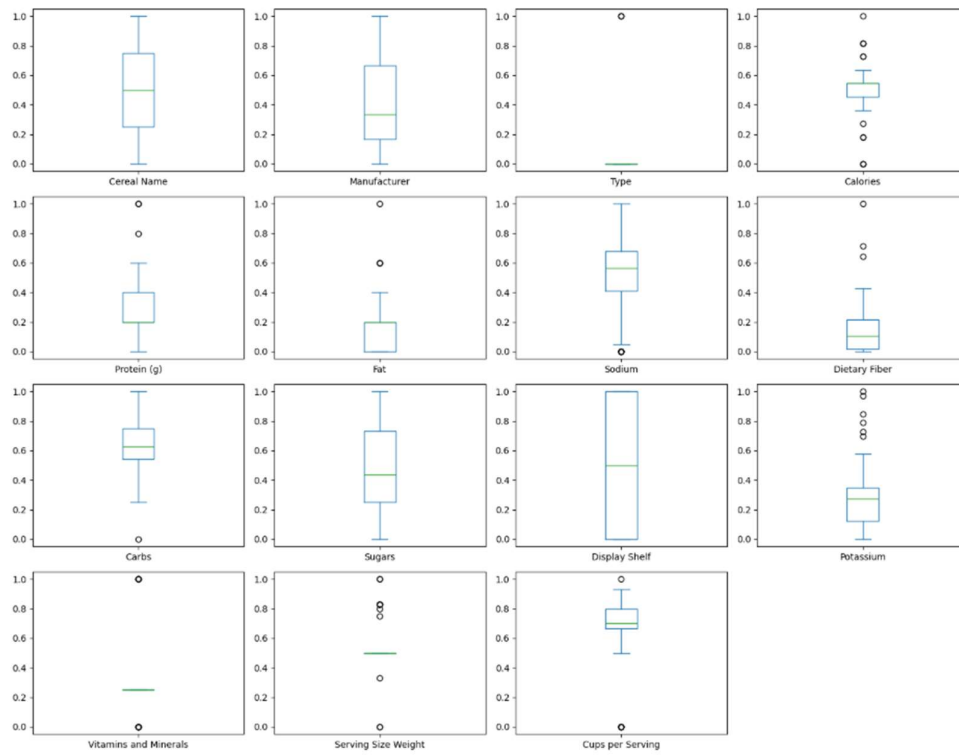
Distribution of Numeric Features



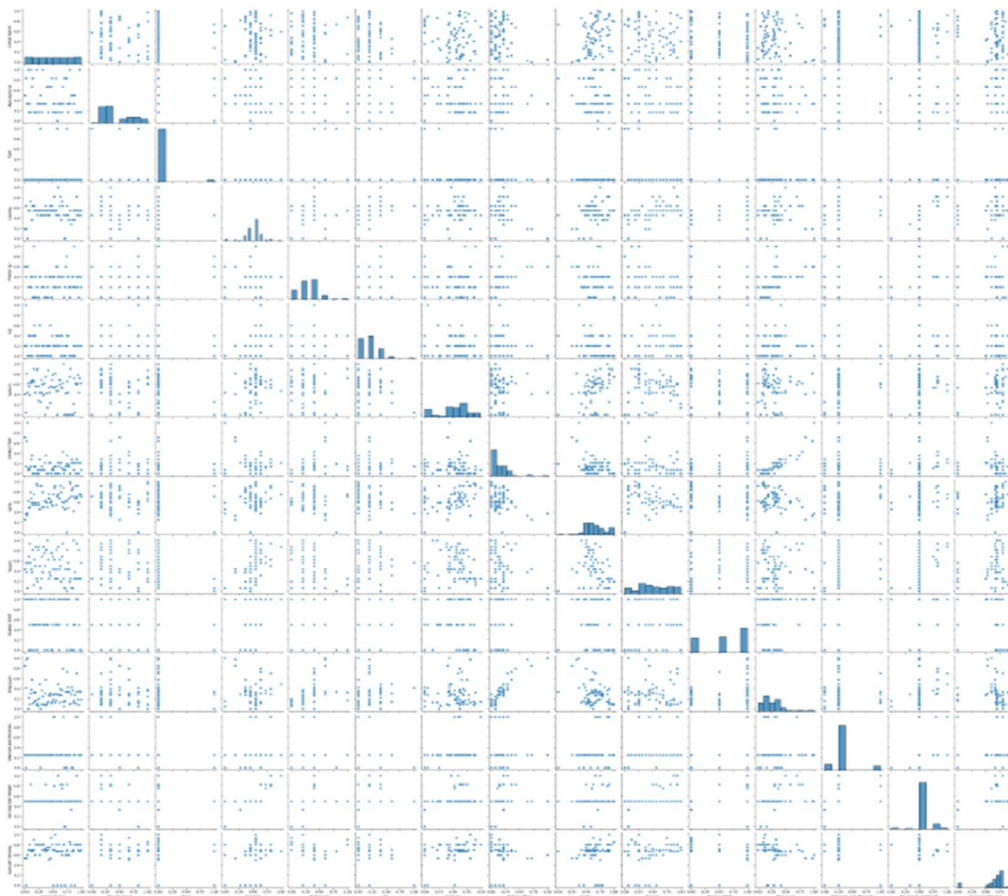
Boxplots for Outlier Detection



Boxplots for Outlier Detection



Boxplot of Nutrient Distribution



Cereal Name → mean: 0.500, median: 0.500, std: 0.295

Manufacturer → mean: 0.437, median: 0.333, std: 0.278

Type → mean: 0.041, median: 0.000, std: 0.199

Calories → mean: 0.505, median: 0.545, std: 0.168

Protein (g) → mean: 0.300, median: 0.200, std: 0.218

Fat → mean: 0.189, median: 0.200, std: 0.193

Sodium → mean: 0.502, median: 0.562, std: 0.266

Dietary Fiber → mean: 0.149, median: 0.107, std: 0.172

Carbs → mean: 0.650, median: 0.625, std: 0.181

Sugars → mean: 0.486, median: 0.438, std: 0.279

Display Shelf → mean: 0.588, median: 0.500, std: 0.417

Potassium → mean: 0.283, median: 0.275, std: 0.213

Vitamins and Minerals → mean: 0.284, median: 0.250, std: 0.228

Serving Size Weight → mean: 0.527, median: 0.500, std: 0.151

Cups per Serving → mean: 0.652, median: 0.700, std: 0.231

## Statistical Analysis / Hypothesis Testing :

Here we are doing 2 tests

```
# =====  
# Hypothesis 1: Testing on Cereal Dataset  
# Tests:  
# 1. Hot vs Cold Calories (t-test)  
# 2. Correlation between Calories and Sugars (Pearson)  
# =====  
# =====  
# Hypothesis 2: Correlation between Calories and Sugars  
# -----  
# H0: No correlation between Calories and Sugars  
# H1: There is a correlation  
# Test: Pearson correlation  
# =====
```

```
[2]: # =====
# 📝 Hypothesis Testing on Cereal Dataset
# Tests:
# 1. Hot vs Cold Calories (t-test)
# 2. Correlation between Calories and Sugars (Pearson)
# =====

import pandas as pd
from scipy import stats

# Load dataset
df = pd.read_csv(r"C:\Users\sripr\Downloads\cereal 1 (1).csv")

# Drop missing values (if any)
df = df.dropna()

# Hypothesis 1
hot_cal = df[df["Type"] == "H"]["Calories"]
cold_cal = df[df["Type"] == "C"]["Calories"]

t_stat, p_val = stats.ttest_ind(hot_cal, cold_cal, equal_var=False)

print("♦ Hypothesis 1: Calories in Hot vs Cold cereals")
print(f"T-statistic = {t_stat:.4f}")
print(f"P-value = {p_val:.4f}")

if p_val < 0.05:
    print("→ Reject H₀: There is a significant difference between Hot and Cold cereals.\n")
else:
    print("→ Fail to reject H₀: No significant difference between Hot and Cold cereals.\n")

# =====
# 📝 Hypothesis 2: Correlation between Calories and Sugars
# -----
# H₀: No correlation between Calories and Sugars
# H₁: There is a correlation
# Test: Pearson correlation
# =====

r, p_corr = stats.pearsonr(df["Calories"], df["Sugars"])

print("♦ Hypothesis 2: Correlation between Calories and Sugars")
print(f"Correlation coefficient (r) = {r:.4f}")
print(f"P-value = {p_corr:.4f}")

if p_corr < 0.05:
    print("→ Reject H₀: Calories and Sugars are significantly correlated.")
else:
    print("→ Fail to reject H₀: No significant correlation found.")

♦ Hypothesis 1: Calories in Hot vs Cold cereals
T-statistic = -2.5886
P-value = 0.0117
→ Reject H₀: There is a significant difference between Hot and Cold cereals.

♦ Hypothesis 2: Correlation between Calories and Sugars
Correlation coefficient (r) = 0.5477
P-value = 0.0000
→ Reject H₀: Calories and Sugars are significantly correlated.
```

Output also attached in above screen shot only

## 1. Introduction / Objective

The project analyzes the nutritional composition of cereals using a dataset from Kaggle. The objective is to explore, visualize, and test relationships between factors like calories, sugar, and cereal type to identify significant patterns and insights.

## 2. Data Collection

The dataset contains **74 records and 15 attributes**, including Calories, Sugars, Fat, Fiber, and Type (Hot or Cold). It provides nutritional information for various cereal brands and categories.

## 3. Data Preprocessing

Data cleaning involved handling missing values, removing duplicates, encoding categorical data, and treating outliers using the IQR method. Numeric features were standardized for consistency, and a cleaned dataset was prepared for analysis.

## 4. Data Visualization & Exploration

Visualizations such as histograms, boxplots, and bar charts were used to explore data distributions.

The correlation heatmap showed strong positive relationships between **Calories, Sugars, and Carbohydrates**.

Hot cereals were observed to have higher calorie values than cold cereals.

## 5. Statistical Analysis / Hypothesis Testing

Two statistical tests were performed:

1. **Independent t-test:** Showed a significant difference in calories between **Hot** and **Cold** cereals ( $p < 0.05$ ).
2. **Pearson correlation:** Revealed a strong positive correlation between **Calories** and **Sugars** ( $p < 0.05$ ).

## 6. Interpretation & Conclusion

The analysis confirms that sugar levels significantly influence calorie content, and hot cereals generally contain more calories than cold ones. Overall, this project demonstrates the complete data analytics process—covering data cleaning, visualization, and hypothesis testing—to derive meaningful nutritional insights.